



PyQuest

Information- & Aufgabenblatt

Kapitel 14: Klassen

Baue eigene Baupläne für Objekte mit eigenen Eigenschaften und Fähigkeiten.

Objekte und Klassen

Eine **Klasse** ist ein **Bauplan** für Objekte. Stell dir einen Keksausstecher vor: Die Klasse ist der Ausstecher, jedes einzelne Objekt ist ein Keks, der damit ausgestochen wurde – gleiche Form, aber jeder Keks für sich.

Eine leere Klasse und zwei Objekte davon:

Beispiel

```
class Hund:
    pass

hund1 = Hund()
hund2 = Hund()
hund1.name = "Bello"
hund2.name = "Rex"
print(hund1.name)
print(hund2.name)
```

class Hund: definiert die Klasse. **pass** heißt "hier kommt (noch) nichts rein" – nur ein Platzhalter. **Hund()** erstellt ein neues **Objekt** (auch "Instanz" genannt).

Jedes Objekt hat seine **eigenen** Attribute: `hund1.name` und `hund2.name` sind unabhängig voneinander.

Aufgabe 1: Was entsteht, wenn man `Hund()` aufruft?

- ☐ a) Ein neues Objekt (Instanz) der Klasse Hund
- ☐ b) Die Klasse Hund wird gelöscht
- ☐ c) Ein Fehler, weil Hund leer ist



Aufgabe 2: Definiere eine leere Klasse Katze (mit pass). Erstelle ein Objekt meine_katze und weise ihm das Attribut name = "Minka" zu. Gib meine_katze.name aus.

Dein Code:

Attribute, Methoden und `__init__`

`__init__` ist eine besondere Methode, die automatisch aufgerufen wird, wenn ein neues Objekt erstellt wird. Damit legst du die **Startwerte** (Attribute) fest, statt sie wie in der letzten Lektion einzeln zuzuweisen.

`__init__` setzt den Namen, `bellen()` ist eine eigene Methode:

Beispiel

```
class Hund:
    def __init__(self, name):
        self.name = name

    def bellen(self):
        print(self.name + " sagt: Wuff!")

hund1 = Hund("Bello")
hund1.bellen()
```



self steht immer als erster Parameter da und bezeichnet "dieses eine Objekt selbst". Beim Aufruf `hund1.bellen()` musst du **self** **nicht** angeben – Python übergibt automatisch `hund1`.

`Hund("Bello")` ruft im Hintergrund `__init__(self, "Bello")` auf und setzt `self.name = "Bello"`.

Aufgabe 3: Wann wird `__init__` aufgerufen?

- ☐ a) Nur wenn man es manuell aufruft
- ☐ b) Automatisch, wenn ein neues Objekt erstellt wird
- ☐ c) Nie, es ist nur Dekoration

Aufgabe 4: Definiere eine Klasse `Person` mit `__init__(self, name, alter)`, die `self.name` und `self.alter` setzt. Füge eine Methode `vorstellen(self)` hinzu, die Ich heiße `NAME` und bin `ALTER` Jahre alt. ausgibt. Erstelle eine `Person` mit Name "Mia" und Alter 17 und rufe `vorstellen()` auf.

Dein Code:

Übung: eine eigene Klasse bauen

Jetzt bringen wir alles zusammen: `__init__` setzt den Startzustand, eine **Methode** kann diesen Zustand später **verändern** – Attribute merken sich das über mehrere Methodenaufrufe hinweg.

Ein einfaches Konto, bei dem `einzahlen()` den Kontostand erhöht:

Beispiel

```
class Konto:
    def __init__(self, kontostand):
        self.kontostand = kontostand

    def einzahlen(self, betrag):
        self.kontostand = self.kontostand + betrag

konto = Konto(100)
```



```
konto.einzahlen(50)
print(konto.kontostand)
```

`konto.einzahlen(50)` verändert `self.kontostand` **dauerhaft** im Objekt `konto`. Ruft man danach `konto.kontostand` ab, ist der neue Wert gespeichert – das Objekt "merkt sich" seinen Zustand.

Aufgabe 5: Warum ist `konto.kontostand` nach `einzahlen()` dauerhaft verändert?

- ☐ a) Weil `self.kontostand` im Objekt gespeichert wird, nicht nur lokal in der Methode
- ☐ b) Ist es nicht, der alte Wert bleibt bestehen
- ☐ c) Weil Python das automatisch zurücksetzt

Aufgabe 6: Definiere eine Klasse `Konto` mit `__init__(self, kontostand)`. Füge eine Methode `abheben(self, betrag)` hinzu, die den Kontostand um `betrag` verringert. Erstelle ein Konto mit Startguthaben 200, hebe 30 ab und gib den Kontostand aus (soll 170 sein).

Dein Code: